TEDLA

Hypertension NLP Pipeline

User Guide & Technical Documentation

Version: 1.0

Platform: Windows 10 / Linux

Language: Python 3.11+

Purpose: Clinical NLP extraction for hypertension research

This document is intended for research teams replicating the TEDLA pipeline.

For internal use only. Do not distribute outside authorized personnel.

Confidential - For internal research use only

Table of Contents

Part I: Quick Start Guide **3**
1. What is TEDLA? 3
2. System Requirements 3
3. Getting Started (Step by Step) 4
4. Input Data Format 5
5. Output Data Format 6
6. Troubleshooting 7

Part II: Technical Documentation **8**
7. Architecture Overview 8
8. Pipeline Components 9
9. NLP Processing Details 11
10. Search Terms (Full List) 12
11. Configuration Reference 13
12. File Structure 14

Part I: Quick Start Guide

1. What is TEDLA?

TEDLA (Text Extraction for Drug and Lab Analysis) is a clinical Natural Language Processing (NLP) pipeline designed to extract hypertension-related information from clinical notes. It reads clinical notes written by doctors and nurses, then automatically finds mentions of symptoms, side effects, and conditions related to high blood pressure.

In simple terms: you give it clinical notes, and it gives you a structured spreadsheet showing which hypertension-related terms were found, whether they were negated (e.g., "no headache" vs "reports headache"), and the surrounding context.

> **Who is this for?**
> This tool is built for research teams at clinical sites who need to replicate the TEDLA hypertension analysis. You do not need programming experience to use it. The graphical interface handles everything.

2. System Requirements

- Python 3.11 or newer installed on your computer
- Windows 10 or later (also works on Linux/Mac)
- At least 4 GB of RAM (8 GB recommended for large datasets)
- At least 2 GB of free disk space for packages and models
- Internet connection (only needed for first-time dependency installation)

> **How to check if Python is installed**
> Open Command Prompt (search "cmd" in Start) and type: python --version
> You should see something like "Python 3.12.0". If not, download Python from python.org.

3. Getting Started (Step by Step)

Step 1: Open the Launcher

Navigate to the "launcher" folder inside the project directory. Double-click run.bat to start the application. A dark-themed window will appear with three main sections: Install, Generate Test Data, and Run Pipeline.

```
Location: tedla-hypertension-project-databricks\launcher\run.bat
```

Step 2: Install Dependencies

Click the blue "Install" button. This will download all required Python packages and the spaCy language model. The log panel at the bottom shows real-time progress. When you see 'All dependencies installed and verified!' in green, you are ready to proceed.

> **Note**
> You only need to install dependencies once. After the first install, you can skip this step in future sessions.

Step 3: Prepare Your Data

You have three options for input data:

Option A: Import a CSV or Excel file (Recommended for new users)

Click the purple "Import CSV/Excel" button in the Run Pipeline section. Select your file containing clinical notes. The file must have at least a "note_text" column. The tool will convert it to the format the pipeline needs and auto-fill the input/output paths.

Option B: Generate synthetic test data

Use the Generate Test Data section. Pick a folder, set how many notes to create (100 is good for testing), and click "Generate". This creates realistic-looking clinical notes to verify the pipeline works correctly.

Option C: Use existing Parquet files

If you already have data in Parquet format from the original TEDLA setup, simply browse to the input folder containing the parquet files.

Step 4: Run the Pipeline

Verify the Input and Output paths are filled in, then click the green "Run Pipeline" button. The progress bar shows how far along the processing is, including an estimated time remaining. The log panel shows detailed output.

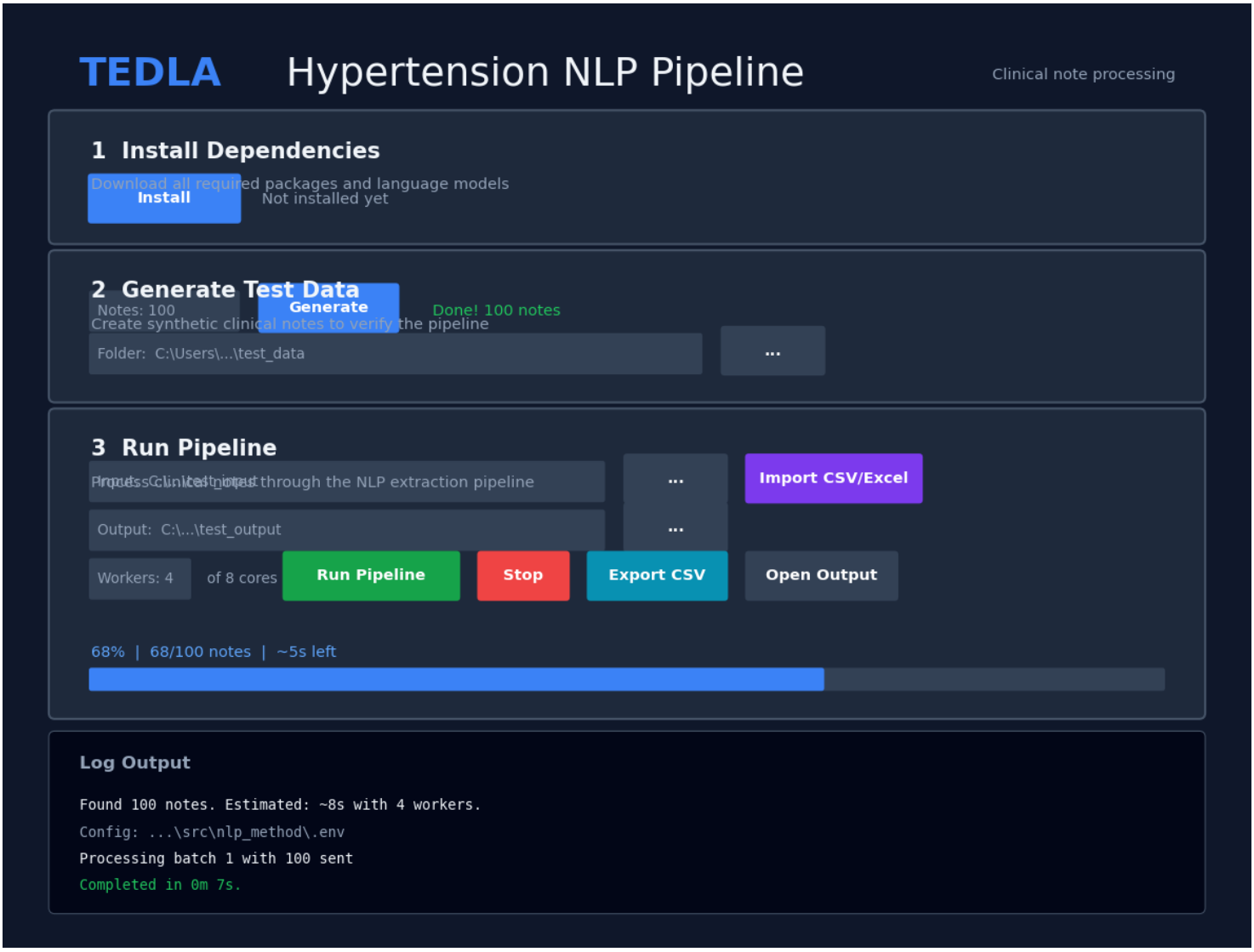
Workers: This controls how many CPU cores are used for parallel processing. The default (4) works well for most computers. Using more workers processes notes faster but uses more memory.

Step 5: Export Results

When the pipeline finishes, the "Export CSV" button becomes available. Click it to save the results as a CSV or Excel file that you can open in Excel, Google Sheets, or any data analysis tool. You can also click "Open Output" to browse the output folder.

Application Interface

Below is the TEDLA launcher interface with all sections labeled:



4. Input Data Format

Input and Output Data Formats

INPUT (CSV / Excel)

note_id	note_text	person_id (optional)	note_date (optional)	note_type (optional)
1001	Patient reports headache...	500001	2023-01-15	outpatient
1002	No evidence of syncope...	500002	2023-02-20	inpatient
1003	Family history of HTN...	500001	2023-03-10	admission

OUTPUT (CSV / Excel)

note_id	search_term	window_text	is_negated	note_type group
1001	headache	...reports headache on...	False	outpatient
1002	syncope	...no evidence of syncope...	True	inpatient
1003	hypertension	...family hx of HTN...	False	admission

CSV / Excel Input

When using the Import CSV/Excel feature, your file needs at minimum one column:

Column Name	Required?	Type	Description
note_text	Yes	Text	The full clinical note text
note_id	No	Number	Unique ID for each note (auto-generated if missing)
person_id	No	Number	Patient ID (auto-generated if missing)
note_date	No	Date	Date of the note (defaults to today if missing)
note_type	No	Text	outpatient, inpatient, admission, etc.

> **Tip**
> The simplest input file is a CSV with just one column: note_text. Everything else is auto-generated. For more accurate results, include note_id, person_id, and note_date.

Parquet Input (Advanced)

The pipeline natively uses Apache Parquet files. If you receive data from the original TEDLA system, it will be in this format. The Import function automatically converts CSV/Excel to Parquet. Five Parquet datasets are required:

Folder	Key Columns	Purpose
note_data/	note_id, note_text	The actual clinical note text
data_to_process/	note_id, person_id, note_date, ...	Metadata about each note
note_to_groups_map/	note_id, 8 boolean columns	Which note type group each note belongs to
note_to_visit_date/	note_id, visit dates	Visit date range information
note_group_to_term/	search_term, 8 flag columns	Which terms apply to which note types

5. Output Data Format

The pipeline produces results in SQLite database files (one per worker). Using the "Export CSV" button consolidates all results into a single file. Each row in the output represents one hypertension-related term found in a clinical note.

Output Columns

Column	Type	Description
note_id	Number	The ID of the clinical note where the term was found
patient_id	Number	The patient this note belongs to
note_date	Date	When the note was written
search_term	Text	The hypertension term that was matched (e.g., 'headache')
window_text	Text	The surrounding text context (20 words before and after)
is_negated	Boolean	True if the term was negated (e.g., 'no headache')
is_patient_communication	Boolean	True if the note is a patient message
is_note_within_visit	Boolean	True if the note falls within a visit window
outpatient_notes_group	Boolean	True if this is an outpatient note
inpatient_notes_group	Boolean	True if this is an inpatient note
admission_notes_group	Boolean	True if this is an admission note
discharge_summary_...	Boolean	True if this is a discharge summary
emergency_dept_...	Boolean	True if this is an ED note
problem_list_...	Boolean	True if from a problem list
ecg_impression_...	Boolean	True if from an ECG impression
communication_...	Boolean	True if from a communication encounter

Understanding Negation

"is_negated = True" means the term was mentioned but denied. For example, "Patient denies headache" would produce: search_term=headache, is_negated=True. This is important for research because negated terms mean the patient does NOT have that symptom.

6. Troubleshooting

Problem: Install button fails with 'pip not found'
Solution: Python may not be properly installed or not in your PATH. Reinstall Python from python.org and make sure to check 'Add Python to PATH' during installation.

Problem: 'ModuleNotFoundError: No module named spacy'
Solution: Dependencies are not installed. Click the Install button first and wait for it to complete.

Problem: 'en_core_web_sm not found'
Solution: The spaCy language model did not download properly. Click Install again to retry.

Problem: Pipeline runs but finds 0 results
Solution: Check that your input data actually contains clinical text. The pipeline looks for specific hypertension-related terms (see Section 10 for the full list).

Problem: 'OSError: No such file or directory'
Solution: The input path is wrong. Make sure the Input field points to a folder that exists and contains the data files.

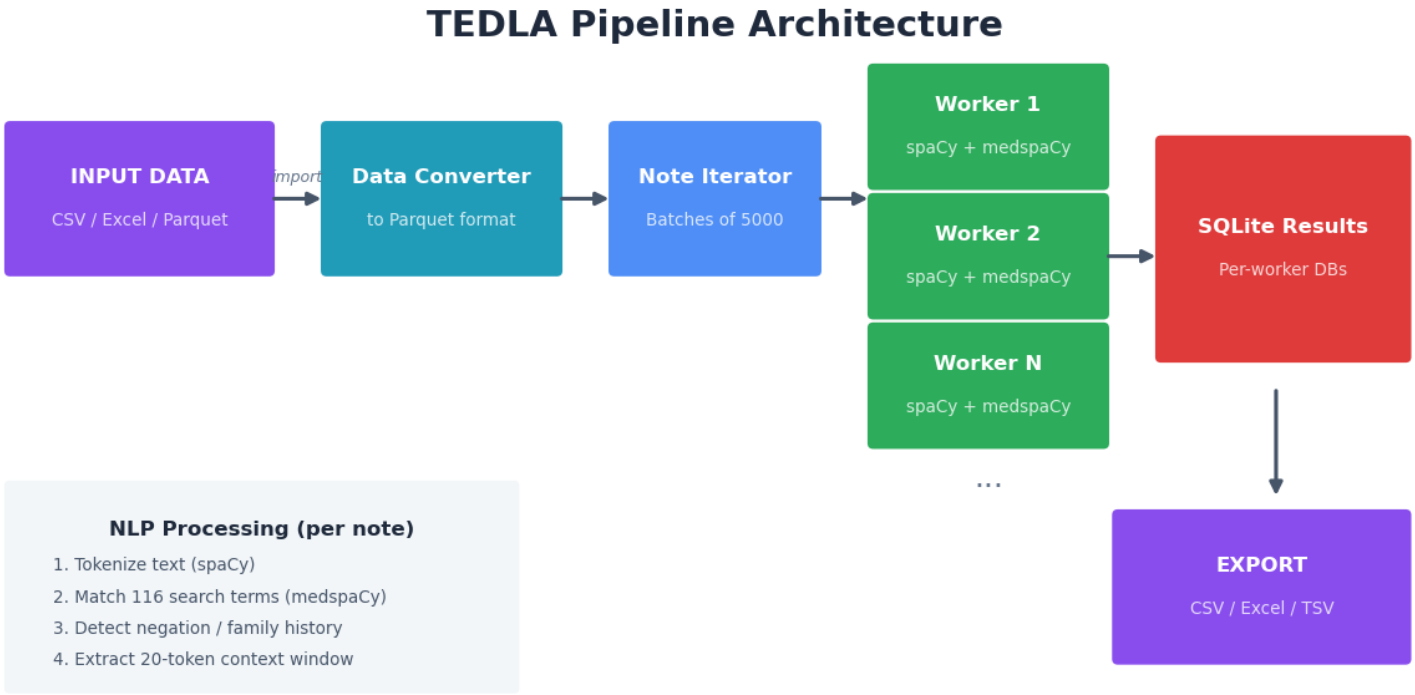
Problem: Very slow processing
Solution: Try increasing the Workers count (up to your CPU core count). Also ensure no other heavy programs are running.

Problem: Import CSV/Excel fails
Solution: Make sure your file has a 'note_text' column header in the first row. Check that the file is not currently open in Excel.

Part II: Technical Documentation

7. Architecture Overview

The TEDLA pipeline uses a producer-consumer multiprocessing architecture. A main process reads clinical notes in batches from Parquet files and places them into a bounded queue. Multiple worker processes consume batches from the queue, process each note through the NLP pipeline, and write results to per-worker SQLite databases.



Key architectural decisions:
- Bounded queue (default size 128) provides backpressure to prevent memory exhaustion
- Per-worker SQLite databases eliminate write contention between processes
- Batch processing (5000 notes per batch) amortizes the overhead of IPC
- Cross-platform file locking ensures safe concurrent database access

8. Pipeline Components (Detailed)

8.1 Entry Point (__main__.py)

Location: src/nlp_method/__main__.py

This is the main orchestrator. It reads configuration from environment variables (loaded from the .env file), creates a multiprocessing Queue, spawns N worker processes, iterates through all notes in batches, feeds them into the queue, then waits for all workers to finish. Finally, it calls cleanup() to close all database connections.

```
  # Simplified flow:
queue = mp.Queue(maxsize=128)
workers = create_nlp_workers(num_workers, queue)
for batch in NotesIterator(note_data, batch_size=5000):
    queue.put(batch)  # blocks if queue is full
for _ in workers:
    queue.put(None)  # sentinel to stop workers
cleanup(queue, processes, databases)
```

8.2 NLP Worker (nlp_worker.py)

Location: src/nlp_method/nlp/nlp_worker.py

Each worker runs in its own OS process. On startup, it initializes a spaCy pipeline, loads the medspaCy components, and signals readiness via a multiprocessing.Event. The main process waits up to 300 seconds for all workers to signal ready. Each worker then enters a loop: pull a batch from the queue, process it, write results, repeat until it receives a None sentinel value.

8.3 NLP Processor (nlp_method.py)

Location: src/nlp_method/nlp/nlp_method.py

The core NLP engine. It loads the en_core_web_sm spaCy model, then adds two medspaCy pipeline components:

1. medspacy_target_matcher - Configured with 116 TargetRule objects, one per hypertension search term. Each rule matches a specific clinical term and labels it as a "SYMPTOM" entity.

2. medspacy_context - The context detection component from medspaCy. It analyzes the linguistic context around each matched entity to determine:
   - Is the mention negated? (e.g., 'no headache', 'denies syncope')
   - Is it about a family member? (e.g., 'mother had hypertension')
   - Is it hypothetical? (e.g., 'if patient develops tachycardia')

8.4 Note Batch Processor (note_batch_processor.py)

Location: src/nlp_method/notes/note_batch_processor.py

This component receives a pandas DataFrame of notes, extracts the text and metadata, passes each note through the NLP Processor, then constructs ResultRecord objects. For each matched entity, it:

1. Identifies the entity span in the text
2. Extracts a context window of 20 tokens before and after the entity
3. Calculates character offsets for precise text extraction
4. Checks the negation status from medspaCy's context component
5. Creates a ResultRecord with all metadata attached

8.5 Notes Iterator (notes_iterator.py)

Location: src/nlp_method/notes/notes_iterator.py

Responsible for loading and joining the input data. It reads note text from note_data parquet files, joins with metadata (data_to_process), note group mappings, and visit dates. The joined data is then yielded in configurable batch sizes (default 5000 notes per batch). A debug mode allows limiting the number of iterations for testing purposes.

8.6 Results Storage

Location: src/nlp_method/results/

Results are stored in SQLite databases, one per worker process. This eliminates write contention that would occur if all workers wrote to a single database. Each database contains a 'results' table with 21 columns (see Section 5 for the full schema). A cross-platform FileLock class provides mutex protection for concurrent access scenarios.

8.7 Data Extraction Layer

Location: src/data_extract/

This layer provides abstraction over the input data sources. Key classes:

- PolarDataSet: Base class for Polars-backed datasets (parquet files)
- NoteData: Reads clinical note text from NOTE_DATA_LOCATION
- NoteMetaData: Reads note metadata from DATA_TO_PROCESS_DATASET
- NoteToGroupsMap: Maps notes to their type groups (8 categories)
- NoteDataAggregator: Joins all data sources into the final DataFrame

9. NLP Processing Details

9.1 Technology Stack

The NLP processing relies on two main libraries:

spaCy (v3.7.5) - An industrial-strength NLP library that provides tokenization, part-of-speech tagging, and named entity recognition. The pipeline uses the en_core_web_sm model (small English model, ~12 MB).

medspaCy (v1.3.1) - A clinical NLP toolkit built on top of spaCy, specifically designed for processing medical text. It provides:

- Target matching for clinical terms
- Context detection (negation, family history, hypothetical)
- Section detection in clinical notes

9.2 Processing Pipeline

Each clinical note goes through these steps:

1. Text Preprocessing: The raw note text is lowercased and stripped of leading/trailing whitespace. This ensures case-insensitive matching.

2. Tokenization (spaCy): The text is split into tokens (words, punctuation). spaCy's tokenizer handles medical abbreviations and special characters.

3. Target Matching (medspaCy): 116 hypertension-related search terms are matched against the tokenized text. Matches are labeled as SYMPTOM entities. The matching is phrase-based, so multi-word terms like 'erectile disfunction' are matched as a unit.

4. Context Detection (medspaCy): For each matched entity, the context component examines surrounding words to determine:

- Negation: triggered by words like 'no', 'denies', 'negative for', 'without'
- Family history: triggered by 'mother', 'father', 'family history of'
- Hypothetical: triggered by 'if', 'should', 'might develop'

5. Context Window Extraction: A window of 20 tokens before and after each entity is extracted. This provides human-readable context for reviewing results.

6. Result Recording: Each match produces a ResultRecord with the term, context window, negation status, character offsets, and all note metadata.

9.3 Note Types (8 Categories)

The pipeline recognizes eight types of clinical notes:

Note Type	Description
Outpatient	Regular clinic visit notes (most common, ~30% of notes)

Inpatient	Hospital stay daily progress notes (~15%)
Admission	Initial hospital admission notes (~10%)
Discharge Summary	Notes written when patient leaves hospital (~15%)
Emergency Department	ER visit documentation (~10%)
Problem List	Active diagnosis and problem lists (~10%)
ECG Impression	Electrocardiogram interpretation notes (~5%)
Communication	Phone calls, messages between patient and provider (~5%)

10. Hypertension Search Terms (Full List)

The pipeline searches for 116 clinical terms related to hypertension, its symptoms, side effects of antihypertensive medications, and related conditions. These terms were curated by clinical domain experts.

1. angioedema
2. ascvd score
3. bradycardia
4. breast enlargement
5. breast pain
6. constipation
7. dec. libido
8. decreased heart rate
9. decreased libido
10. dizziness
11. doe
12. drowsiness
13. dry cough
14. dry mouth
15. dryness of the mouth
16. dyspnea on exertion
17. ed
18. elevated hr
19. erectile disfunction
20. excessive facial hair
21. excessive sleepiness
22. face swelling
23. facial erythema
24. falling out
25. fatigue
26. flushing
27. gingival hypertrophy
28. gout
29. gouty flare
30. gum swelling
31. gynecomastia
32. h/a
33. ha
34. hallucinations
35. headache
36. heart failure with preserved ejection fract
37. heart failure with reduced ejection fraction
38. hepatotoxicity
39. hfpef
40. hfref
41. high ca
42. high calcium
43. hirsutism
44. hyper ca
45. hyper k
46. hyper ua
47. hypercalcemia
48. hyperkalemia
49. hyperprolactinemia
50. hyperuricemia
51. hypokalemia
52. hyponatremia
53. impotence
54. inc hr
55. inc. wt.
56. increase in exercise
57. increased heart rate
58. increased lfts
59. increased liver function tests
60. increased prolactin
61. increased weight
62. insomnia
63. light headedness
64. lip swelling
65. liver toxicity
66. low alcohol intake
67. low blood pressure
68. low bp
69. low hr
70. low k
71. low na
72. low salt intake
73. low sodium
74. lower k
75. lupus sx
76. lupus symptom
77. lupus-like syndrome
78. mastodynia
79. medication adherence
80. n/v
81. nausea
82. nightmare
83. nonproductive cough
84. orthostatic hypotension
85. parasomnia
86. parkinsonian symptoms
87. parkinsonism
88. passed out
89. physical activity
90. poor sleep
91. positive ana
92. positive hgc
93. pregnancy
94. pregnant
95. sexual se
96. sexual side effect
97. shortness of breath
98. sick sinus syndrome
99. sob
100. somnolence
101. symptoms of parkinson
102. syncope
103. tachycardia
104. tired
105. tongue swelling
106. ua high
107. urgency
108. uric acid elevated
109. urinary urgency
110. vivid dreams
111. vomiting
112. weight gain
113. weight loss
114. weight reduction
115. hypo k

11. Configuration Reference

The pipeline is configured through environment variables, stored in an .env file. The launcher GUI generates this file automatically. Here is the full reference:

Variable	Default	Description
NUM_WORKER_PROCESSES	4	Number of parallel worker processes
NOTE_ITERATOR_BATCH_SIZE	5000	Notes per batch sent to workers
MAX_WORKER_QUEUE_SIZE	128	Maximum batches in the queue
WORKER_READY_TIMEOUT_SECONDS	300	Seconds to wait for workers to start
WORKER_JOIN_TIMEOUT_SECONDS	3600	Seconds to wait for workers to finish
SPACY_MODEL	en_core_web_sr	spaCy language model to use
NOTE_DATA_LOCATION	(required)	Path to folder with note_data parquet files
DATA_TO_PROCESS_DATASET	(required)	Path to folder with metadata parquet files
SQLITE_DB_PATH	(required)	Path where result databases are saved
LOG_PATH	(required)	Path for log files
RESULTS_TABLE_NAME	results	Name of SQLite results table (min 5 chars)

12. Project File Structure

```
tedla-hypertension-project-databricks/
|
|-- launcher/
|   |-- launcher.py          # GUI application (all-in-one)
|   |-- run.bat              # Windows double-click launcher
|
|-- src/
|   |-- nlp_method/
|   |   |-- __main__.py      # Pipeline entry point
|   |   |-- nlp/
|   |   |   |-- nlp_method.py    # spaCy + medspaCy processor
|   |   |   |-- nlp_worker.py    # Worker process logic
|   |   |   |-- initializer.py   # Worker initialization
|   |   |-- notes/
|   |   |   |-- notes_iterator.py      # Batch data loading
|   |   |   |-- note_batch_processor.py # Per-batch NLP processing
|   |   |-- results/
|   |   |   |-- result_record.py        # Result data model
|   |   |   |-- result_writer.py        # Write results to DB
|   |   |   |-- FileLock.py             # Cross-platform locking
|   |   |   |-- datastore/
|   |   |       |-- results_data_store.py  # SQLite operations
|   |   |       |-- location.py            # DB path management
|   |   |-- data/
|   |       |-- map_search_term_to_note_group.py
```

```
|   |       |-- map_search_term_to_note_group.csv
|   |
|   |-- data_extract/
|   |   |-- datasources/
|   |       |-- __init__.py          # PolarDataSet base class
|   |       |-- data/note_data.py    # Note text loader
|   |       |-- data/data_to_process.py  # Metadata loader
|   |       |-- note_data_aggration.py   # Data join logic
|   |
|   |-- common/
|       |-- worker_mixin.py  # Worker ID mixin
|
|-- .scripts/
|   |-- search_terms.txt     # 116 search terms
|
|-- requirements.txt         # Python dependencies
|-- TEDLA_Pipeline_Documentation.pdf  # This document
```

End of Documentation

For questions or support, contact your site coordinator.

TEDLA Hypertension NLP Pipeline - Version 1.0